

Team Narwal

Sprint 6



Inhoud Meeting

- Waar staan we nu
- Terugblik:
 - Wat is er behaald
 - Wat is er individueel gebeurd
 - Waar liepen we tegen aan
- Vooruitblik:
 - Wat gaan we doen
 - Hoe gaan we dat bereiken
- Terugkoppeling



Waar staan we nu

Fase 1

Kop: Erin & Joni

- Milestone 1: Vorm
- Milestone 2: Veranderen
- Milestone 3: Bewegingen/Draaien
- Milestone 4: Samenvoegen

Segmenten: Jente & Tim

- Milestone 1: Vorm
- Milestone 2: Veranderen
- Milestone 3: Bewegingen

Basis: Adriaan

- Milestone 1: Vorm
- Milestone 2: Veranderen
- Milestone 3: Bewegingen/Draaien

Fase 2

Inverse Kinematica: Tim & Joni

- Onderzoek
- Toepassing
- Forward kinematica
- Samenvoegen

Vision: Erin & Adriaan

- Yolo training data
- Camera implementatie
- Samenvoegen

PID: Jente

- PID uitwerken
- Samenvoegen

Vooruitblik

Wat gaan we doen

Vision: Adriaan & Erin

- Implementatie van camera
- Begin opencv
- Implementatie valve mesh



PID: Jente

- De PID fine tuning
- Kameleon



Inverse Kinematica: Tim & Joni

- Onderzoek
- Inverse Kinematica in code
- Forward Kinematica



Joni

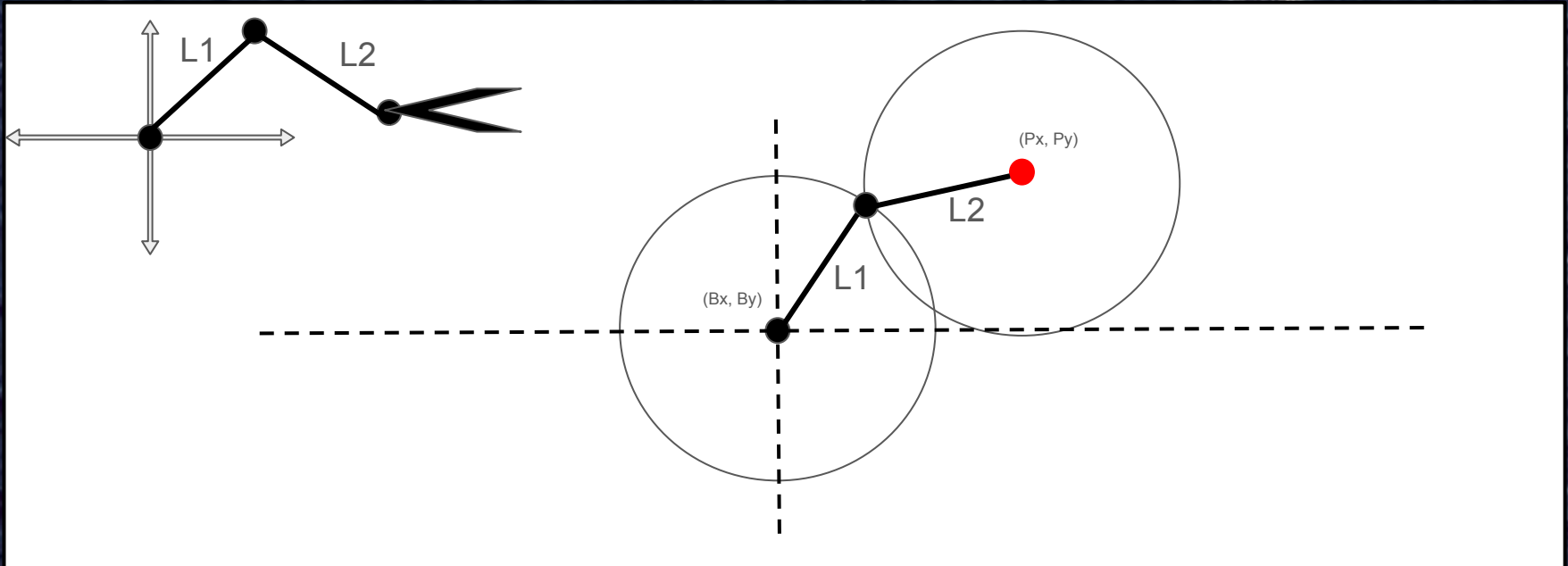
- Onderzoek Inverse Kinematics
- Inverse Kinematics in code

- Analytisch
- Numeriek
- Circle Intersection



Joni

Circle Intersection

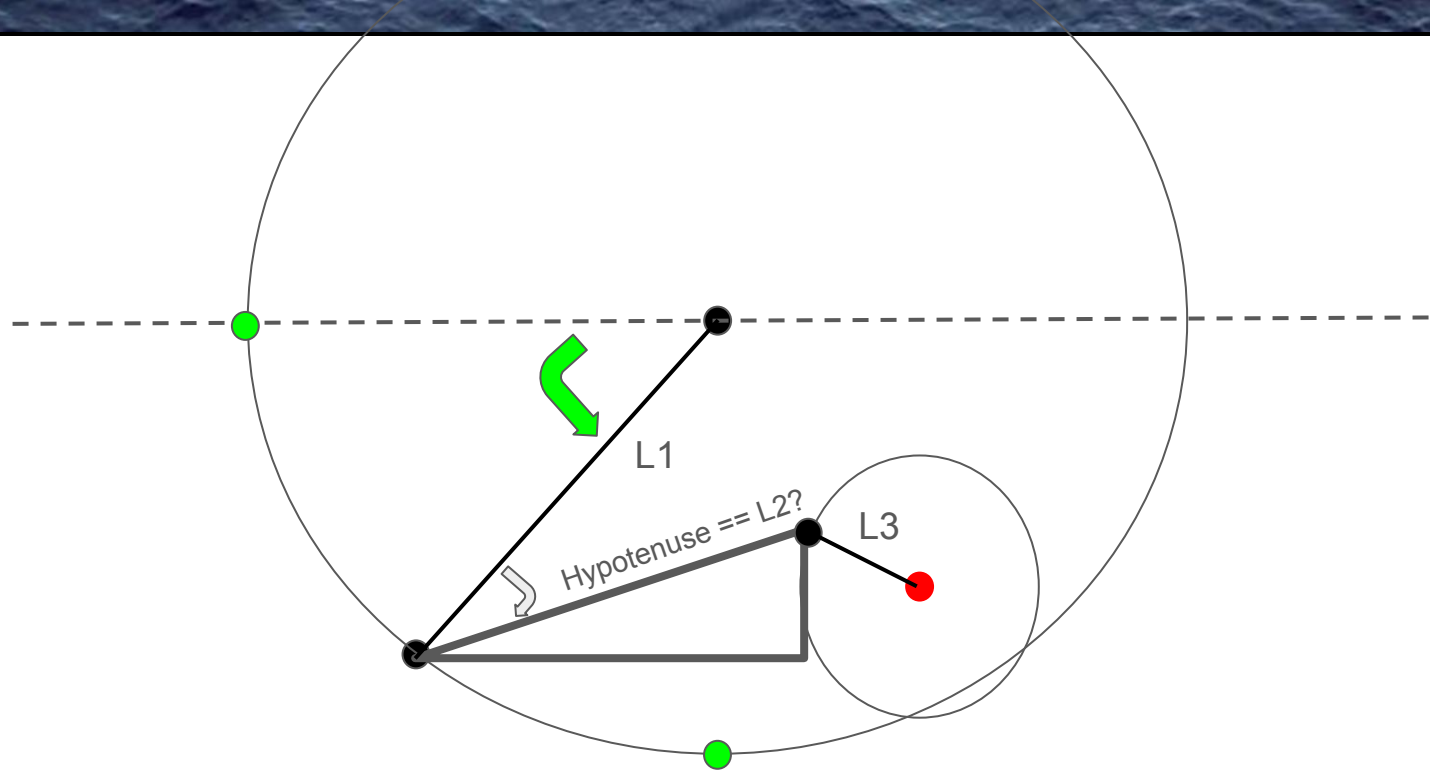


Joni

Code

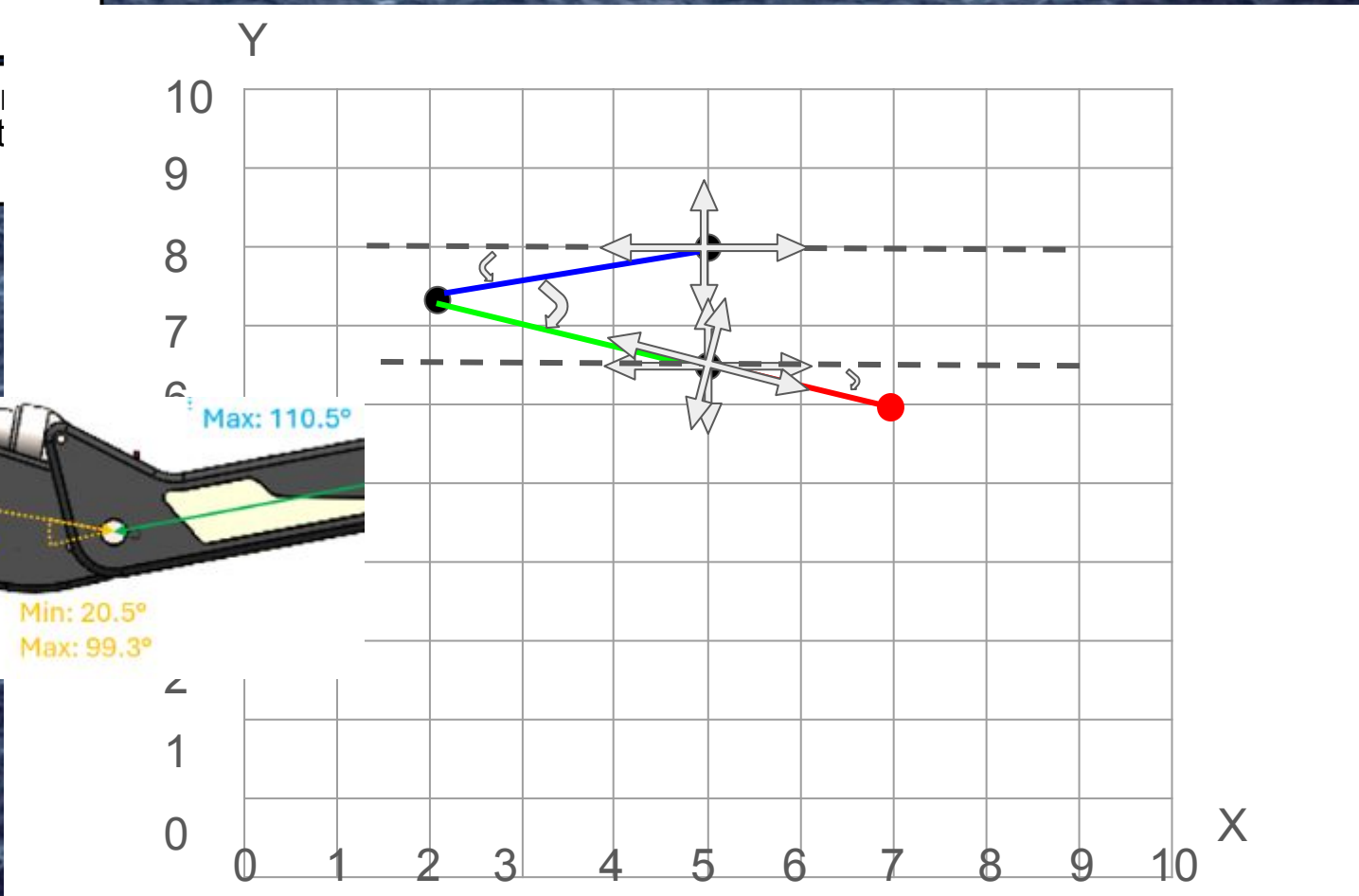
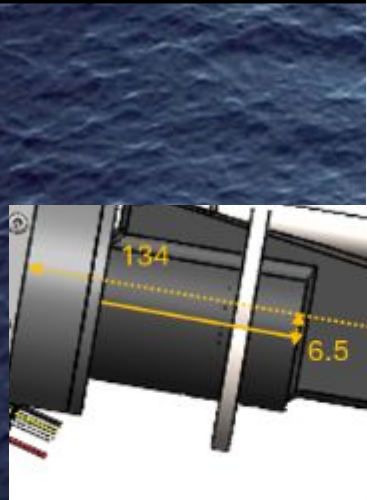
```
3
4 def circleEF(x, y, l):
5     coordinates = []
6     i = 0.0
7     while i < 360.0:
8         cor = []
9         cor.append(x + l * math.cos(math.radians(i)))
10        cor.append(y + l * math.sin(math.radians(i)))
11        coordinates.append(cor)
12        i += 0.5
13    return coordinates
14
15 def ki(x1, y1, x2, y2, l1, l2, l3, A1):
16     coordinates = circleEF(x2, y2, l3)
17
```





Joni

- Onderzoek Inverse Kinematik
- Inverse Kinematik



Joni

Code



```
15 def ki(x1, y1, x2, y2, l1, l2, l3, A1):
16     coordinates = circleEF(x2, y2, l3)
17
18     i = 0
19     while i + A1 < 90.0:
20         new_x = x1 + l1 * math.cos(math.radians( 180 + i ))
21         new_y = y1 + l1 * math.sin(math.radians( 180 + i ))
22
23         for c in coordinates:
24             distance = math.sqrt(abs(new_x - c[0])**2 + abs(new_y - c[1])**2 )
25             if abs(distance - l2) <= 0.05:
26
27                 across = math.sqrt( abs(x1 - c[0])**2 + abs(y1 - c[1])**2 )
28                 devide = (2*l1*l2)
29                 angle = ( (l1**2 + l2**2 - across**2)/devide )
30                 angle = max(-1.0, min(1.0, angle))
31                 anglefinal = math.acos(angle)
32
33                 if math.degrees(anglefinal) <= 85:
34                     xb = c[0] - new_x
35                     yb = c[1] - new_y
36                     rotation = math.atan2(yb,xb)
37
38                     minangle = 20 + math.degrees(rotation)
39                     maxangle = - 99 + math.degrees(rotation)
40
41                     angleEndeffectorr = math.atan2(y2 - c[1], x2 - c[0])
42                     if math.degrees(angleEndeffectorr) <= minangle and math.degrees(angleEndeffectorr) >= maxangle:
43                         print("mogelijk")
44                         print(f'lenght L2: {distance}')
45                         print("rotatie:", math.degrees(rotation))
46
47                         print("\n")
48                         print(f'min angle: {minangle}')
49                         print(f'max angle: {maxangle}')
50                         print(f'angle of l3 from x axis: {math.degrees(angleEndeffectorr)}')
51
52                         print("\n")
53                         print(f'x2 {new_x}')
54                         print(f'y2 {new_y}')
55                         print(f'x3 {c[0]}')
56                         print(f'y3 {c[1]}')
57                         print(f'xp {x2}')
58                         print(f'xy {y2}')
59
60                     return i, math.degrees(anglefinal), math.degrees(angleEndeffectorr)
```


Joni

Code

```
PS C:\Users\jonik\TISEM4\personal-JFKernel\Groepsproject\code> python ikk.py
mogelijk
lenght L2: 3.0493902365301206
rotatie: -13.059835488490316

min angle: 6.940164511509684
max angle: -112.05983548849032
angle of l3 from x axis: -15.500000000000004

x2 2.102222521132795
y2 7.223542864692438
x3 5.072739093582754
y3 6.534476752156514
xp 7
xy 6
Success! Angles are: 15.0, 28.311020843770336, -15.500000000000004
```

Tim

- Inverse Kinematics in code (testversie)
- Voor nu radialen gebruikt, maar moeten worden omgerekend naar graden.

```
# Cosine
cos_q2 = ((x**2 + y**2 - B**2 - C**2) / (2 * B * C))
```

```
q2 = math.acos(cos_q2)
q2_alt = -math.acos(cos_q2)
```

```
# Components
k1 = B + C * math.cos(q2)
k2 = C * math.sin(q2)
```

```
k1_alt = B + C * math.cos(q2_alt)
k2_alt = C * math.sin(q2_alt)
```

```
# Correction
q1 = math.atan2(y, x) - math.atan2(k2, k1)
q1_alt = math.atan2(y, x) - math.atan2(k2_alt, k1_alt)
```

```
return q1, q1_alt, q2, q2_alt
```

```
-0.43591187238263895
1.8679016930200774
1.7853937568297442
-1.8679016930200774
```

```
1 import math
2 import os
3 import numpy as np
4
5 def inverse_kinematics(x, y, B, C):
6
7     # JOINTS = {
8     # "base": "robosub_base_to_robosub_rotating_base",
9     # "upper_arm": "robosub_rotating_base_to_upper_arm_segment",
10    # "lower_arm": "upper_arm_segment_to_lower_arm_segment",
11    # "palm": "lower_arm_segment_to_palm_segment",
12    # "claw_rot": "palm_segment_to_rotating_base_claw",
13    # "upper_finger": "rotating_base_claw_to_upper_finger_claw",
14    # "lower_finger": "rotating_base_claw_to_lower_finger_claw"
15    # }
16
17    # /cmd/vel
18
19    r = math.sqrt(x**2 + y**2)
20
21    if r > (B + C):
22        raise ValueError()
23
24    if r < abs(B - C):
25        raise ValueError()
26
```

```
# Calculated angles
q1, q1_alt, q2, q2_alt = inverse_kinematics(2.5, 2.0, 2.3, 3.0)
print(q1)
print(q2)

def send_angle(topic, angle):
    cmd = f"gz topic -t {topic} -m gz.msgs.Double -p 'data: {angle}'"
    os.system(cmd)
```

```
send_angle("/arm/shoulder_cmd", q1)
send_angle("/arm/elbow_cmd", q2_alt)
```

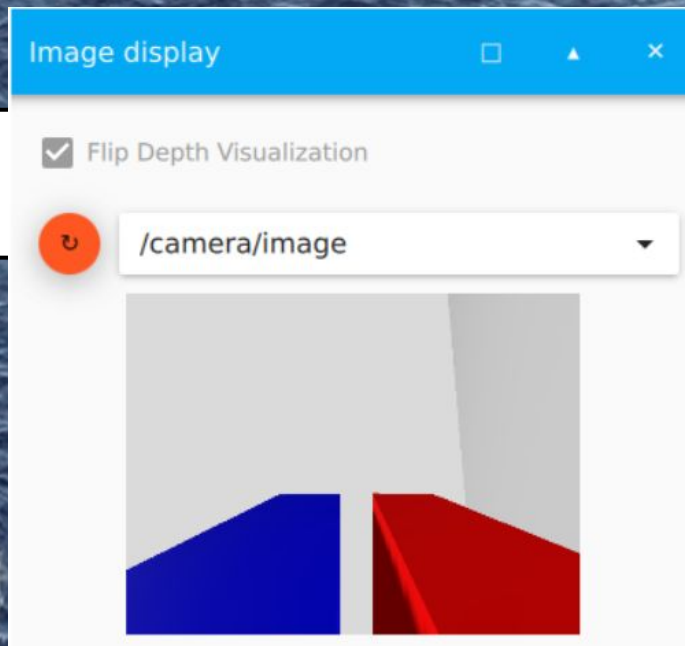
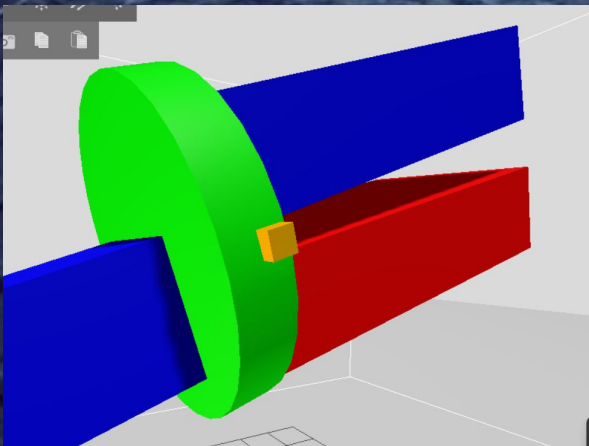
Adriaan

- Onderzoek AI
- Het maken van de AI



Erin

- Implementatie camera
- Omzetten van datastream naar afbeelding



```
<link name="camera_link">
  <pose>0 0 0 0 0 0</pose>

  <inertial>
    <mass>0.5</mass>
    <inertia>
      <ixx>0.001</ixx>
      <iyy>0.001</iyy>
      <izz>0.001</izz>
    </inertia>
  </inertial>

  <sensor type="camera" name="my_sensor">
    <camera>
      <horizontal_fov>1.047</horizontal_fov>
      <image>
        <width>640</width>
        <height>480</height>
        <format>R8G8B8</format>
      </image>
      <clip>
        <near>0.1</near>
        <far>100</far>
      </clip>
    </camera>
    <always_on>true</always_on>
    <update_rate>30</update_rate>
  </sensor>
</link>
```

```
<plugin filename="gz-sim-sensors-system" name="gz::sim::systems::Sensors" />
```

Doelen:

Inverse Kinematica: Tim & Joni

- Onderzoek afronden
- Samenvoegen
- Forward kinematica

Vision: Erin & Adriaan

- Yolo training data
- Camera implementatie

PID: Jente

- Samenvoegen

Documentatie!!

Feedback

Eindproduct

- Oriëntatiedocument
- Onderzoeken
- Code

